

27 | 跨端的机遇：小程序、Flutter和React Native原理对比

2022-7-1 蒋宏伟



你好，我是蒋宏伟。从今天起，我们进入了一个新的篇章，新架构原理篇。

在此之前，我们所学的核心基础篇、社区生态篇、基础设施建设篇都是贴近应用层面的知识，而在新架构原理篇中，我们会重点学习偏底层的纯技术类知识。

看到这儿你可能会问了：“我们在日常工作中很难用得到框架原理呀，学习这个到底有什么意义呢？”

这个问题问得非常好，其实我们换个角度来看就豁然开朗了：“如果只学那些日常会用到的、别人也会知识点，我的职业竞争力在哪里？”

掌握那些大家都会的应用层面的知识只是及格，要想更进一步，我们就要有更高的技术追求。我认为学习底层框架的意义不在于“内卷”，也不在于为了面试去背八股文，**为的是解决别人解决不了的问题，为的是“创新”。**

“人无我有，人有我优”，才是你的核心竞争力。

不过，在深入学习 React Native 新架构的原理细节之前，我想先带你对比下小程序、Flutter、React Native 之间的架构，让你先从宏观上理解各大框架的架构差异，为后续深入原理打下基础。

小程序

我们先来看小程序。

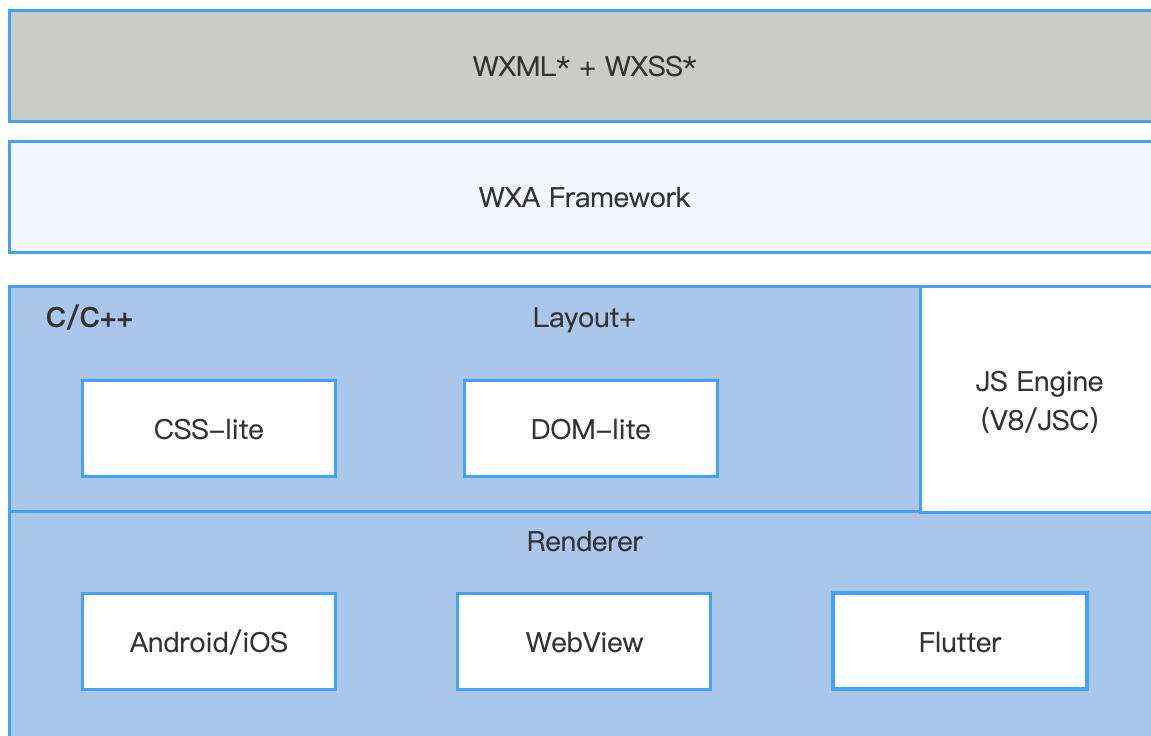
从技术上说，小程序是在一个应用程序中再嵌入另一个应用程序，也就是说微信本身就是一个应用程序，而小程序就是微信应用程序之上的程序，这在国内独有现象，国外是没有的。

从商业角度讲，这进一步把微信应用，打造成了一个操作系统平台之上的平台，或者 AppStore 之上的 AppStore。相对于自己开发原生应用而言，由于微信等超级 App 在国内的普及程度非常高，开发者不仅可以使⽤这些超级 App 提供的技术能力和触达用户的渠道，还能轻易地实现跨端，节约成本。

小程序虽好，但它是一个闭源软件，而不是一个开源软件，我们没办法直接抄作业，只能从官方的文章中借鉴和分析。

以微信小程序为例，在 [《基于小程序技术栈的微信客户端跨平台实践》](#) 这篇文章中，微信技术团队分享了微信小程序在各个技术方向的跨端探索，包括基于 WebView 的、基于 RN-like 的、基于 Flutter 的，这三种跨端思路。

微信小程序的探索



极客时间

第一种是基于 WebView 实现跨端的思路。

但在基于这种思路开发时，微信团队遇到了 WebView 渲染和原生渲染体验不一致的问题。

他们举了两个例子。一个是两个技术方案渲染出来的字体不一致：在 Android 平台上，WebView 没办法和 Android 系统的字体保持一致，体验上有“较为明显的割裂感”。

第二个例子是，在图片和视频混排的场景下，Android 中低端机会出现掉帧的现象。

既然纯 WebView 会存在字体不一致和掉帧的问题，那我们能不能把原生字体、原生组件引进来，把 WebView 中有问题的字体和组件替换掉呢？

这就是第二种思路，使用部分原生组件代替部分 WebView 组件的方案。

因为这套思路，受到的是 React Native 这类框架的启发，因此他们内部也叫做 RN-like 方案。

我们简单说一下这个方案的原理。首先微信小程序提供了自己的一套开发框架和语言，包括用于描述页面基础组件的 WXML（WeiXin Markup Language），以及用于描述页面样式的

WXSS (WeiXin Style Sheets) 。

当 WXML/WXSS 发生改变，也就是 UI 页面发生改变的时候，小程序前端公共库（WXA Framework）会进行一些内部运算，以操作指令的形式将变化结果提交给 C++，具体的布局计算、CSS 样式更新和 DOM 结构变化都是由 C++ 这一层实现的。C++ 计算完成后，再决定是调用 WebView 渲染组件，还是调用原生视图渲染组件。

此外，在微信推出的[同层渲染](#)方案中，就是用原生组件把 video、map、camera、textarea、input 等 WebView 标签替代了。

除了以上两种方案之外，微信还尝试了第三种思路。**第三种思路就是，用 Flutter 完全代替 WebView。**

Flutter 版的小程序的原理大致是，前面部分和 RN-like 方案很像，用小程序前端公共库（WXA）计算 WXML/WXSS 的变化，并将这些变化描述成指令传给 C++ 层。和 RN-like 方案不同的是，C++ 层完成布局计算后，调用的是 Dart2C++ 接口，并将渲染命令传给 Flutter，由 Flutter 进行绘制。

讲完这三种实现思路后，微信技术团队也给出了[三种技术路线的性能对比](#)：

	RN-like vs WebView	Flutter vs WebView
测试环境	Google Pixel 2 XL @Android 7.1 微信 7.0.3 Debug APK 表情商店(小程序版)	Google Pixel XL @Android 9.0 微信 Android 7.0.5 Debug APK Flutter Release Build/AOT 附近的餐厅(小程序)
内存占用	下降 14%~23%	下降 20%
CPU 占用	-	平均 CPU 占用下降 6%
FPS	提升 ~27%	提升 ~10%

你可以看到，相比原始的 WebView 方案，使用 RN-like 和 Flutter 渲染方案后，内存都有所下降，FPS 也都有所提升。

但微信团队综合对比下来，认为目前 RN-like 方案，也就是使用 WebView 加上部分原生组件渲染页面的方案，有极大的灵活性和前端兼容性，并不会用 Flutter 方案将其代替。

Flutter

聊完小程序，接着我们再来聊一下 Flutter。

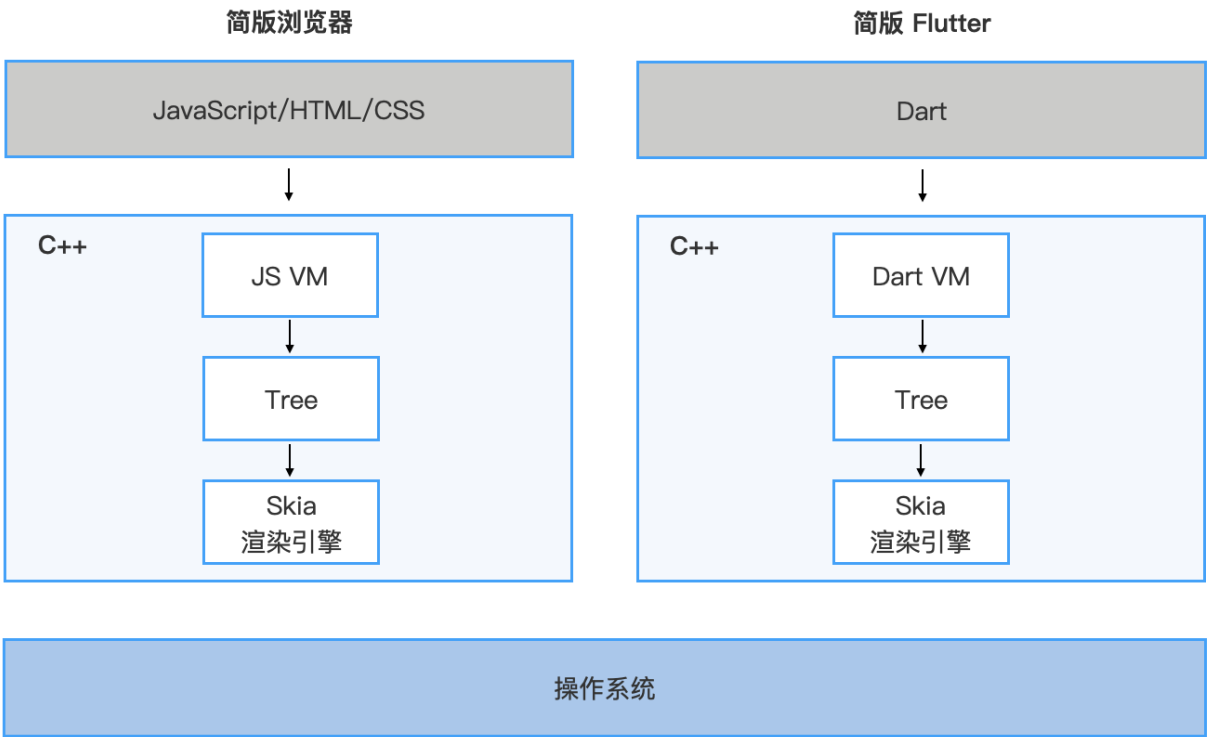
有些 React Native 开发者可能认为，跨端框架了解一个就够了，会 React Native 了就不用再会 Flutter 了。

并不是这样的。即便只开发 React Native，你也应该对竞品框架有所了解，特别是在深入底层原理学习的时候，如果你能把各个框架之间原理对比着学习，会更容易看到它们的相同之处和不同之处，也更容易从宏观层面对它们进行抽象理解。

回到Flutter框架本身。谷歌对 Flutter 的投入力度非常大，因为它不仅是一个跨端框架，还是开发 Fuchsia 操作系统的 UI 界面的工具。

在移动互联网爆发后，Chrome 不仅要支持桌面应用，还要支持移动应用。Flutter 的创始人 Eric Seidel 对 Chrome 进行了拆分，去掉桌面应用的历史包袱后，发现一些核心指标比原来快上了 20 倍，于是就有了后来 Flutter 的故事。

Flutter 现在的定位是一个跨平台 UI 开发工具（UI toolkit），但当你剖析它的原理时，你会发现 Flutter 和浏览器有很多相似之处，你可以相互对照着进行理解：



我们不考虑架构细节，只从最基本的原理上观察，你会发现Flutter 和浏览器的架构非常相似。

首先，Flutter 和浏览器都是跨平台的，而且它们主要用的渲染引擎都是 Skia。

其次，在语言层面上，Flutter 业务层的语言采用的是 Dart，浏览器用的是 JavaScript。Dart 语言和 JavaScript 非常类似，因为 Dart 诞生之初的目标就是想替代 JavaScript。而在框架底层，二者都是用 C++。

第三，在和原生应用通讯上，Flutter 提供了两套通信机制，消息通道（EventChannel）和接口通道（MethodChannel）。消息通道的原理类似于 Hybrid 应用或 React Native 老架构的“bridge”，跨语言通讯的消息都需要进行序列化和反序列化。接口通道的原理类似 JavaScript 直接调用 C++，利用的是引擎提供的脚本语言和 C++ 之间调用能力。

举个例子，如果Flutter 要调用原生能力，比如微信支付能力，实际上它还是通过 Dart 调用消息通道和接口通道，再间接地调用 Java/OC，由 Java/OC 再调用微信提供的原生 SDK。在这种混合应用场景中，Flutter 也没办法避免通信带来的性能折损。

当然，Flutter 和浏览器也有很多不同之处：

Flutter 没有历史包袱。它不需要支持那些不常用功能和布局能力；

Flutter 用的是 Dart，浏览器用的是 HTML/CSS/JavaScript；

Flutter 为了保证 UI 组件（Widgets）的灵活性，使用的是 Dart 来实现的，而浏览器标签提供的 UI 组件能力是由 C++ 实现的，等等。

我认为，Flutter 的优势在于它的纯 Flutter 应用的渲染路径很短。

纯 Flutter 的渲染只涉及独立渲染进程，从业务语言 Dart 的调用，到 C++ 层的渲染和布局，再到调用使用 C++ 写的 Skia 渲染引擎，由 Skia 将计算出来的向量图，栅格化为手机屏幕能够显示位图，也就是我们看到的页面。

而Flutter 的主要槽点是 Dart。

虽然 Dart 语法和 JavaScript 语法很像，但终归是两门语言，增加了很多学习成本和开发成本。Dart 发明之初是为下一代 Web 应用准备的，也有各种 JavaScript 和 Dart 的对比文章

在吹捧 Dart，甚至把 Dart 称为“JavaScript 杀手”。但最终，开发者还是用脚投票选择了 JavaScript 而不是 Dart，导致 Chrome 团队内置 Dart 虚拟机的计划腹死胎中。

其次，由于苹果政策的限制，Dart 语言是不能热更新的。虽然业内也有很多 Flutter 热更新的动态化框架，但这些都是以牺牲 Flutter 性能为基础的。苹果政策规定只有使用 JavaScriptCore 作为引擎的应用才能动态更新，因此 Flutter 的热更新要先执行 JavaScript，再调用 Java/OC，然后才开始执行 Dart，这样 Flutter 的调用栈就变长、变复杂了。

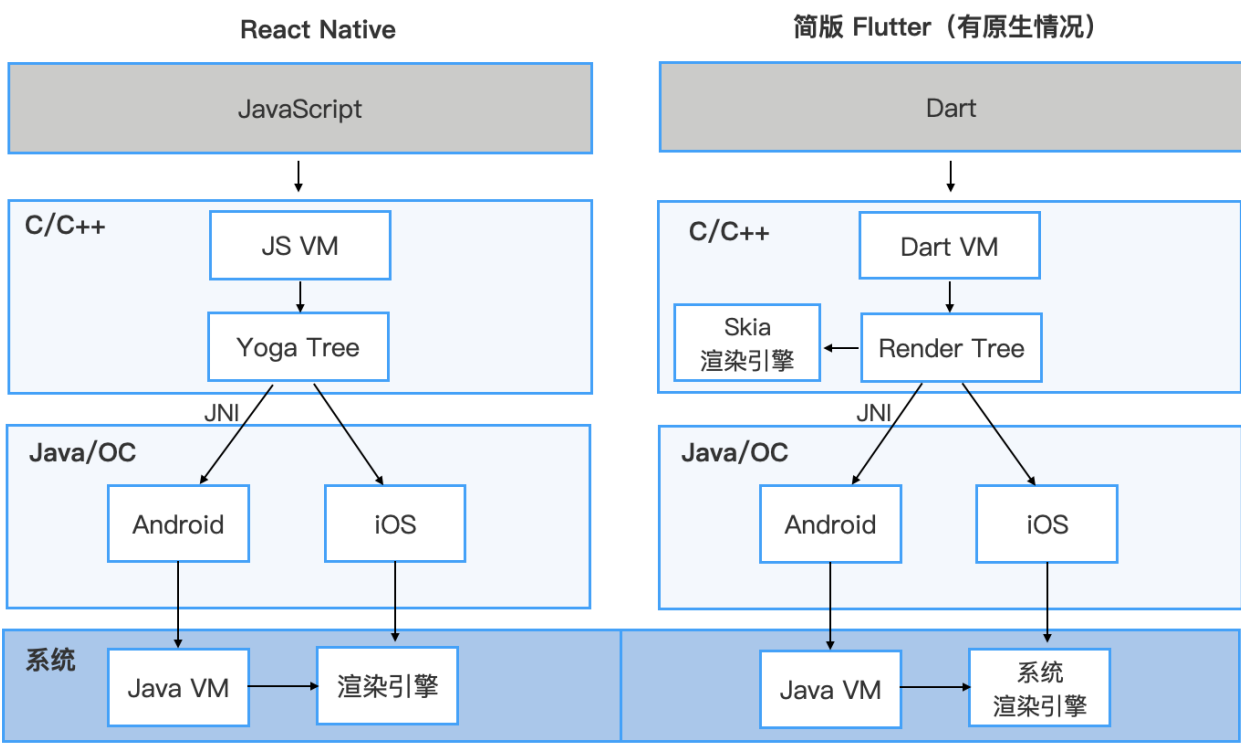
React Native

最后就是我们的主角 React Native 了。

和小程序、Flutter 一样，React Native 也是移动浪潮下的产物，不同的是它走的技术路线是 React + Native 的组合路线，这让 Web 开发者上手非常快。

对于 React Native 框架的整体原理，相信你已经有所了解了，因此我们来综合对比一下。

先来看看 React Native 和 Flutter 的相似之处和不同之处：



二者的相似之处有三点。

首先，[“Flutter 模型架构的灵感来自 Facebook 的 React 框架”](#)，它们都是组件化的、（伪）声明式的。

其次，两个跨端框架对原生开发都有很高的依赖，只要稍微复杂点，都离不开原生的支持，比如电商交易离不开微信支付，在线直播离不开[声网](#)等。

第三点，也是最关键的一点，在混合应用中，由于二者都高度依赖原生开发，因此它们常常需要使用原生应用或操作系统本身提供的组件或 API。咱们不看具体的细节，只从宏观角度看，你会发现无论是 React Native 还是 Flutter，它们使用原生组件或 API 的调用路径是一样的。

上图的左边是 React Native 的渲染路径图，从图中你可以看出来，React Native 最上层的业务语言是 JavaScript，下一层是 C++。C++ 的核心功能主要是布局计算，并且起到了黏合剂的作用，连接了 JavaScript（JS VM）和 Java（JNI）/OC。通过 Java 和 OC 对操作系统提供的原生组件进行增删改查，最后由操作系统调用渲染引擎将原生组件渲染出来。

上图的右边是 Flutter 使用原生组件时的渲染路径图，你可以看到，Flutter 最上层的业务语言是 Dart，下一层也是 C++。C++ 的核心功能也是布局计算和黏合剂，由于是原生组件所以不会使用 Skia 渲染引擎，而是继续调用 Java/OC，最后也是由系统调用渲染引擎才能渲染原生组件。

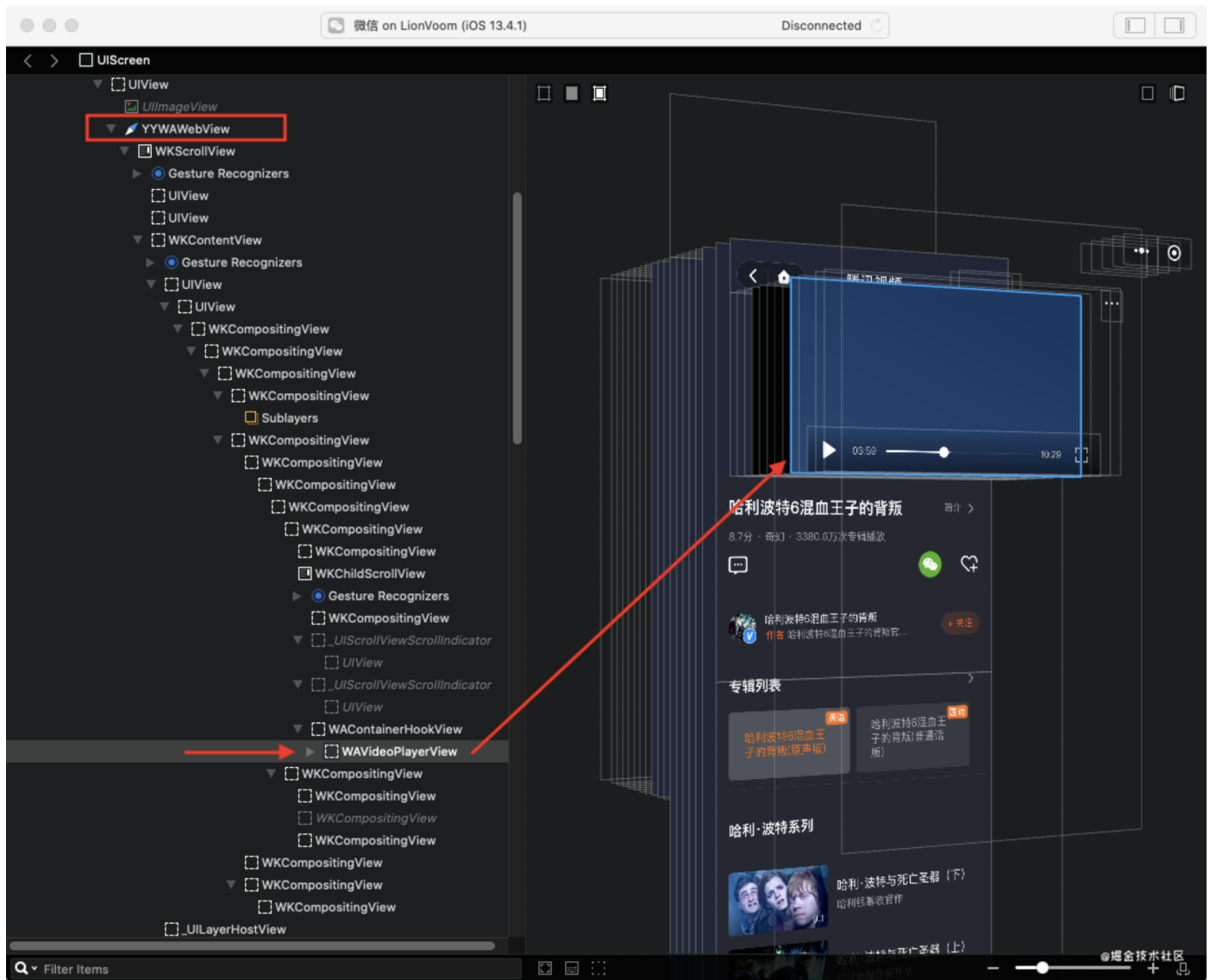
说完相同之处，再来说说二者的不同之处，不同之处我主要介绍两点。

我们再看一次上面这张图。第一个不同点，React Native 用的是 JavaScript，Flutter 用的是 Dart。

第二点，二者的渲染引擎不一样。React Native 用的是操作系统提供渲染引擎，渲染引擎只有一个。但Flutter 用了两个渲染引擎，分别是 Skia 渲染引擎和操作系统提供渲染引擎，那两个渲染引擎应该怎么合作，渲染出同一个页面呢？

还记得微信小程序中的 RN-like 方案吗？我们前面说微信小程序中同时使用了 WebView 和 Native 视图，并且将 WebView 和 Native 视图渲染到了同一个层级，而 WebView 常用的渲染引擎就是 Skia。

你可以看下下面[这张图](#)：



要将 Flutter 的 Skia 渲染引擎和系统渲染引擎绘制在同一个页面，和小程序将 WebView 和 Native 视图渲染到同一个页面的原理是相似的。Flutter 官方给出的原话是：

Hybrid composition appends the native android.view.View to the view hierarchy.

混合集成模式会将原生的 android.view.View 附加到视图层次结构中。

用大白话说就是，在 Flutter 同时使用两个渲染引擎时，会将系统引擎渲染出来的原生视图放到 Skia 渲染出来的视图层级之中。而且 Flutter 官方也承认，使用[混合集成模式渲染原生视图](#)时，会有线程竞争、显存会被复制两次、浪费显存和绘图性能等问题。

由于 React Native 只使用系统渲染引擎，就不存在混合集成模式这种说法，所有的视图都是原生视图，这就简单很多了。

总结

时代在发展，技术也会跟着时代脉搏同时演进，而那些卓越创造者们，无一不从前人的经验中吸取着养分。

在移动浪潮之下，诞生了小程序、Flutter、React Native 等非常多的优秀的跨端框架，是它们给我们大前端开发者带来了便利，也是它们给用户带来了更好体验。

小程序的 XWML 和 WXSS 类似于裁剪的 HTML/CSS，同时它的底层架构又参考了很多 PWA 和 React Native 的思想。更有意思的是，小程序不仅是一个跨端框架，还是一个取得巨大成功的商业平台，把用户和我们这些开发者连接得更紧密了。

Flutter 的创始人本身就是从 Chrome 团队出来的，你也可以从 Flutter 的架构思想上看到它与 Chrome 的很多相似之处，Flutter 不仅是成功的跨端框架，还是为 Fuchsia OS 铺平道路的技术创新。

而 React Native 是一个纯粹的技术框架，它采取了更加社区化的运作方式，微软、Expo、Callstack 都参与了 React Native 的共建。更关键的是，它把 Web 很多规范、编程思想带到了移动端，并且在技术上掀起了一波又一波的移动跨端的探索。



对于我们个人而言，没有什么比直接吸取前人经验能带给我们更好的成长了。技术的浪潮一波接着一波，大的浪潮是大的机遇，小浪潮是小的机遇，我们应该尝试去抓住那一拨属于自己的机遇。

创新往大了说，可以是为业内独立开辟一条新道路跨端框架，比如说小程序、React Native、Flutter 那样有巨大的影响力框架。

也可以是一套内部公司的跨端框架，能够解决公司业务的实际痛点的跨端框架，比如 58RN、MFlutter；还可以是屏蔽平台差异性的开源框架，让所有开源社区受益的 Taro、UniApp 等等。

创新往小了说，可以是利用框架原理解决一个其他同事解决不了的问题；可以帮助团队其他成员提升效率的工具；也可以是 Github 上一个百赞的开源仓库。

但不懂技术原理，只懂如何使用，是没办法抓住技术带来的机遇的。因此，让我们期待接下来 React Native 新架构原理篇的学习吧。

思考题

请你从业务技术选型的角度，分析一下小程序、Flutter、React Native 各自的优缺点。欢迎在留言区分享，我们下一讲见。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

- 上一篇 总结与展望 | 新架构探索之旅，马上开始
- 下一篇 28 | 追本溯源：从第一版 React Native 开始学会读懂源码

精选留言 5